HW 3 Image Clustering Report
Registered Name on Miner: Cryingwalrus65
V Scores for Iris Dataset: 94% for Images Dataset: 75%
By: Pranav Tummalapalli

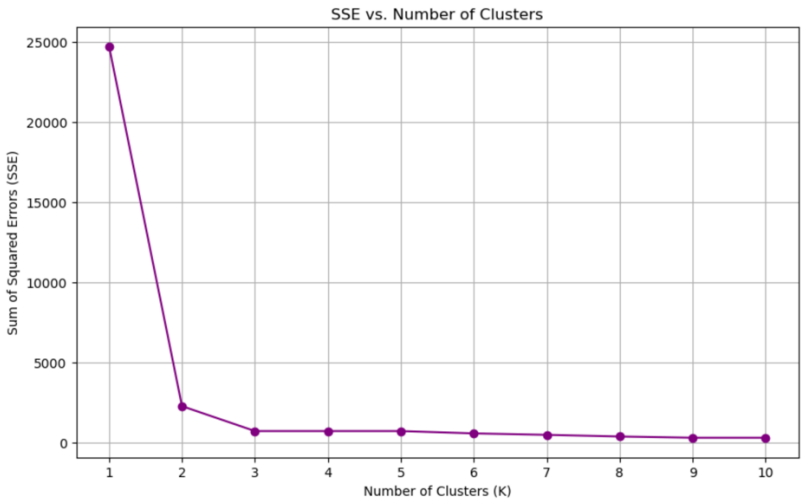
Documentation Steps:

- Data Preprocessing
    - In this step-in order to minimize computation time dimensionality reduction as well as normalization was conducted. In terms of normalization the standard normalize function was called due to the import statement: from sklearn.preprocessing import normalize. Normalization was utilized due to scale uniformity, and overally algorithmic increase in performance. This is especially because when computing distances in a kmeans algorithm each feature contributes equally to the distance computation, thereby creating a more accurate model.
    - Further, in terms of dimensionality reduction TSNE was used with the following parameters n_components=2, random_state=42, perplexity=30, n_iter=800. TSNE also known as t-distributed Stochastic Neighbor Embedding (TSNE). TSNE was utilized opposite of PCA as it was a non-linear dimensionality reduction technique. The parameters n_components specify the number of dimensions you want to reduce your data to, for example, since the data is to be visualized in a 2d space, the n_components parameter is set to 2. Random state is just a random initialization of points the number 42 was selected as it is a commonplace number in the industry. Perplexity refers to the balance between local and global aspects, in short, in larger datasets a larger perplexity is used, and the opposite is true for smaller datasets. Since the perplexity needs to be utilized for both the iris and images datasets both large and small, perplexity was selected to be in the middle to work for both datasets. Finally, the same logic was used for the number of iterations.
- Kmeans
    - In this step the first step was the initialization of the centroids. To create reproducibility a numpy random seeds number was used to get the same data each time. The centroids were created with a low value of the minimum value of self.data, and the high value of the maximum value of self.data with a size equating to the shape of the output array of centroids. Self.data.shape() gives the amount of features. While self.k gives the number of rows in the output array.

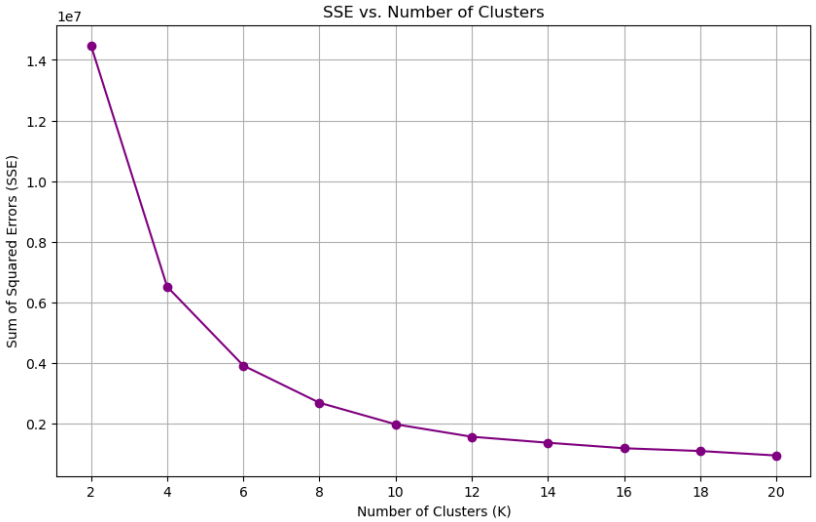
```
[[ 1.36172719  4.85638285]
 [13.65075854  0.36066745]
 [-6.15089997 -5.29210429]]
```

```
[[-18.60411292  58.99412191]
 [ 30.36492547   9.9591213 ]]
[[-18.60411292  58.99412191]
 [ 30.36492547   9.9591213 ]
 [-48.54025005 -51.69595261]
 [-61.95676388  47.21948704]]
[[-18.60411292  58.99412191]
 [ 30.36492547   9.9591213 ]
 [-48.54025005 -51.69595261]
 [-61.95676388  47.21948704]
 [ 12.43529389  25.19852314]
 [-67.09391901  61.66771381]]
[[-18.60411292  58.99412191]
 [ 30.36492547   9.9591213 ]
 [-48.54025005 -51.69595261]
 [-61.95676388  47.21948704]
 [ 12.43529389  25.19852314]
 [-67.09391901  61.66771381]
```

- **Example centroids of Images and Iris**
    - Then the distance was calculated, to calculate the distance of each point to all the centroids, the concept of Pythagoras theorem was used thereby utilizing Euclidean distance which is conducted by square rooting the sum of the delta of the centroids to the data points.
    - Then after the instantiation of the distances and the centroids each data point is assigned to a cluster, represented by the nearest centroid. Then each centroid is updated by calculating the mean distance of all the data points from the centroids cluster. When the update of the centroid doesn't change the value, then the algorithm is finished.
    - Then the SSE is calculated which is sum of squared error. This was modeled after the equation from kmeans slides. This equation comprised of the summation of another summation of the squared distance. This was replicated as the points inside the cluster found by using the cluster indices in the self.data were subtracting with the centroids. Then the difference was squared, thereby creating squared distance which was summed by the variable sse.
    - Clusters were chosen randomly, and the number of iterations were done until the old centroid and new centroids matched perfectly and empty clusters were handled as centroids from the clusters were only updated with the mean if the cluster is greater than 0.
- Plots



- **Iris Datasets' SSE vs. Number of Clusters**



- **Image Datasets' SSE vs. Number of Clusters**

- In these plots we see the SSE value go down as number of clusters (K) increases